



CSC232 - INFORMATION SECURITY

Lab Assignment 03 Report



Submitted By:

Muhammad Rizwan Shafiq

SP24-BCS-069

BCS-4B

Submitted To:

Sir Sabghat Ullah

Table of Contents

Question 1 – Diffie–Hellman Key Exchange	2
Fast Modular Exponentiation	2
Diffie–Hellman Key Exchange Process.....	2
Screenshot of Q1 Output:	2
Question 2 – Man-in-the-Middle Attack on DH.....	2
(i) Is MITM possible on DH? Why?	2
(ii) Can MITM be prevented? How?	3
Analysis Paragraph	3
Screenshot of Q2 Output:	3
Question 3 – ElGamal Over Integers.....	3
Summary	3
Screenshot of Q3 Output:	4
Question 4 – ElGamal Attack (Reused k)	4
Why Reusing k is Dangerous?	4
Screenshot of Q4 Output:	5
Question 5 – Hybrid DH + ElGamal System.....	5
Security Advantages	5
1. Diffie–Hellman – Session Key Generation	5
2. XOR Masking.....	5
3. ElGamal Public-Key Encryption	5
Outcome	5
Screenshot of Q5 Output:	6

Question 1 – Diffie–Hellman Key Exchange

Fast Modular Exponentiation

The Diffie–Hellman protocol uses the square-and-multiply method to efficiently compute

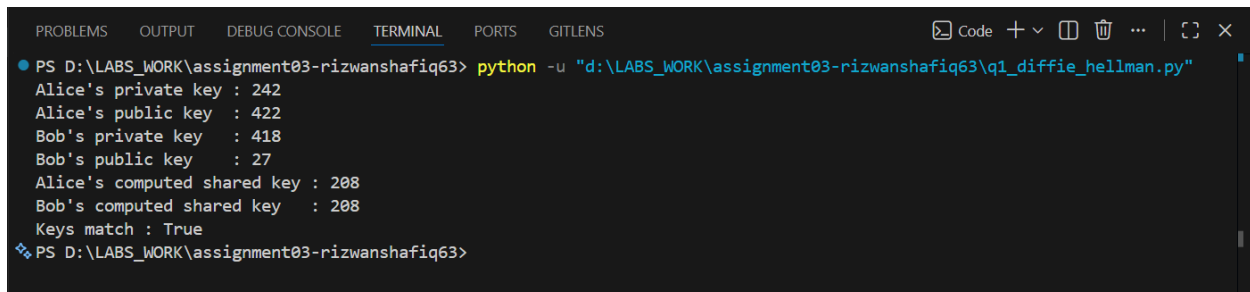
$$g^x \bmod p$$

in $O(\log x)$ time. This method works by repeatedly squaring the base and multiplying only when the current bit of the exponent is 1.

Diffie–Hellman Key Exchange Process

1. Alice chooses private key a ; Bob chooses private key b .
2. They compute public values:
 - $A = g^a \bmod p$
 - $B = g^b \bmod p$
3. Shared keys:
 - Alice computes $K = B^a \bmod p$
 - Bob computes $K = A^b \bmod p$
4. Both obtain the same shared key.

Screenshot of Q1 Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS D:\LABS_WORK\assignment03-rizwanshafi63> python -u "d:\LABS_WORK\assignment03-rizwanshafi63\q1_diffie_hellman.py"
Alice's private key : 242
Alice's public key : 422
Bob's private key : 418
Bob's public key : 27
Alice's computed shared key : 208
Bob's computed shared key : 208
Keys match : True
PS D:\LABS_WORK\assignment03-rizwanshafi63>
```

Question 2 – Man-in-the-Middle Attack on DH

(i) Is MITM possible on DH? Why?

Yes. DH does not authenticate public keys. An attacker can intercept A and B , replace them with her own values, and establish two independent shared keys—one with Alice and one with Bob.

(ii) Can MITM be prevented? How?

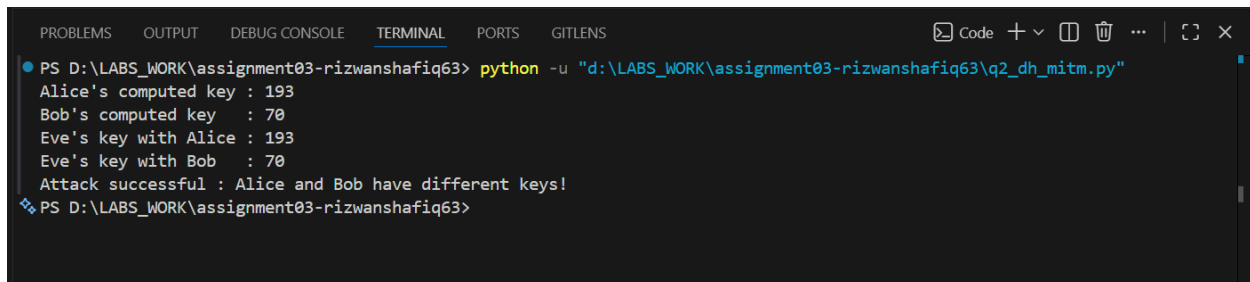
Yes, by authenticating DH messages using:

- Digital signatures
- Certificates (PKI)
- Pre-shared keys
- Authenticated key exchange protocols

Analysis Paragraph

Diffie–Hellman is vulnerable to a man-in-the-middle (MITM) attack without authentication because Alice and Bob have no way to verify that the public values they receive (A and B) actually come from each other; an attacker can intercept these values, replace them with her own, and secretly establish two separate shared keys, one with Alice and one with Bob, while Alice and Bob falsely believe they share a key directly. This can be prevented by adding an authentication mechanism such as digital signatures or certificates (PKI), so that each party signs its DH public value and the other side verifies the signature using a trusted public key. To make DH secure in practice, we would modify the protocol so that Alice and Bob first exchange and verify each other's public keys or certificates, then send their DH public values along with digital signatures over those values; only if the signatures verify correctly do they accept the DH values and compute the shared key, otherwise they abort the key exchange.

Screenshot of Q2 Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS D:\LABS_WORK\assignment03-rizwanshafi63> python -u "d:\LABS_WORK\assignment03-rizwanshafi63\q2_dh_mitm.py"
Alice's computed key : 193
Bob's computed key   : 70
Eve's key with Alice : 193
Eve's key with Bob   : 70
Attack successful : Alice and Bob have different keys!
PS D:\LABS_WORK\assignment03-rizwanshafi63>
```

Question 3 – ElGamal Over Integers

Summary

ElGamal encryption works by selecting a random k for each message and computing ciphertext:

$$\begin{aligned}C_1 &= g^k \bmod p \\C_2 &= m \cdot h^k \bmod p\end{aligned}$$

Decryption uses:

$$m = C_2 \cdot (C_1^x)^{-1} \bmod p$$

Each character of the plaintext message "RETREAT AT 0900" is encrypted individually using its ASCII value.

Screenshot of Q3 Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
PS D:\LABS_WORK\assignment03-rizwanshafi63> python -u "d:\LABS_WORK\assignment03-rizwanshafi63\q3_elgamal.py"
Private key x : 26927

Original message: RETREAT AT 0900

Ciphertexts (per character):
char[0] = 'R' -> (c1=31503, c2=10667)
char[1] = 'E' -> (c1=22294, c2=12985)
char[2] = 'T' -> (c1=8995, c2=18355)
char[3] = 'R' -> (c1=10846, c2=23533)
char[4] = 'E' -> (c1=7976, c2=13798)
char[5] = 'A' -> (c1=24902, c2=11243)
char[6] = 'T' -> (c1=31600, c2=13143)
char[7] = ' ' -> (c1=28618, c2=4242)
char[8] = 'A' -> (c1=5610, c2=13765)
char[9] = 'T' -> (c1=15641, c2=10203)
char[10] = ' ' -> (c1=22657, c2=29458)
char[11] = '0' -> (c1=20892, c2=29493)
char[12] = '9' -> (c1=8758, c2=13676)
char[13] = '0' -> (c1=2084, c2=5187)
char[14] = '0' -> (c1=13147, c2=29768)

Decrypted message: RETREAT AT 0900

Decryption successful: True
❖ PS D:\LABS_WORK\assignment03-rizwanshafi63>

```

Question 4 – ElGamal Attack (Reused k)

Why Reusing k is Dangerous?

ElGamal relies on a **fresh random k** for every encryption. If the same *k* is reused:

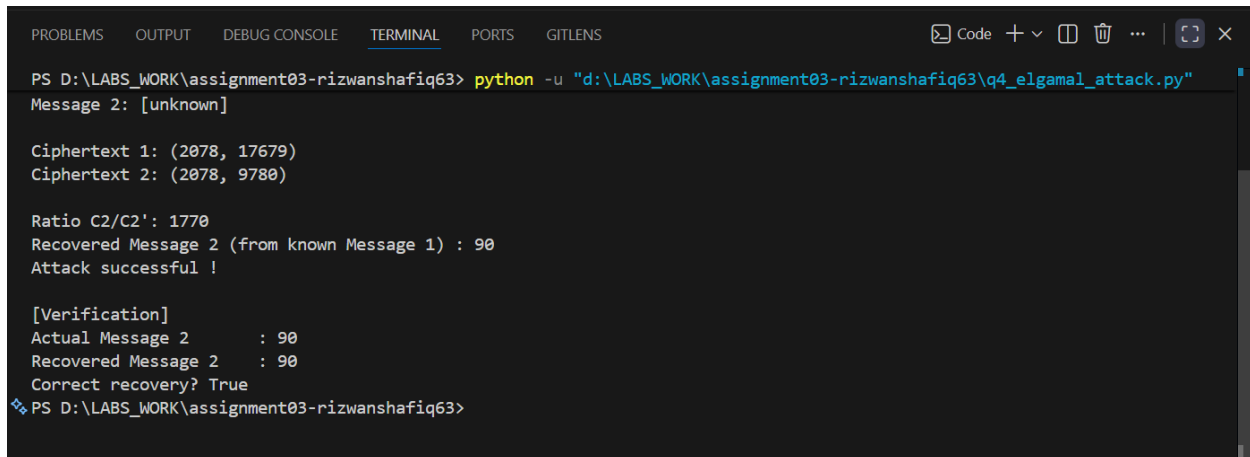
- $C_1 = g^k \bmod p$ will be identical across ciphertexts.
- This allows attackers to compute:

$$\frac{C_{2,1}}{C_{2,2}} = \frac{m_1}{m_2} \bmod p$$

- If one plaintext is known, the other plaintext can be recovered without the private key.

This breaks confidentiality completely.

Screenshot of Q4 Output:



```
PS D:\LABS_WORK\assignment03-rizwanshafi63> python -u "d:\LABS_WORK\assignment03-rizwanshafi63\q4_elgamal_attack.py"
Message 2: [unknown]

Ciphertext 1: (2078, 17679)
Ciphertext 2: (2078, 9780)

Ratio C2/C2': 1770
Recovered Message 2 (from known Message 1) : 90
Attack successful !

[Verification]
Actual Message 2      : 90
Recovered Message 2   : 90
Correct recovery? True
PS D:\LABS_WORK\assignment03-rizwanshafi63>
```

Question 5 – Hybrid DH + ElGamal System

Security Advantages

The hybrid system combines two strong primitives:

1. Diffie–Hellman – Session Key Generation

- Provides a fresh shared key K_{DH} per session
- Offers forward secrecy
- No pre-shared key required

2. XOR Masking

- The plaintext is masked with K_{DH} , making it look random before ElGamal encryption
- Prevents direct exposure of plaintext patterns

3. ElGamal Public-Key Encryption

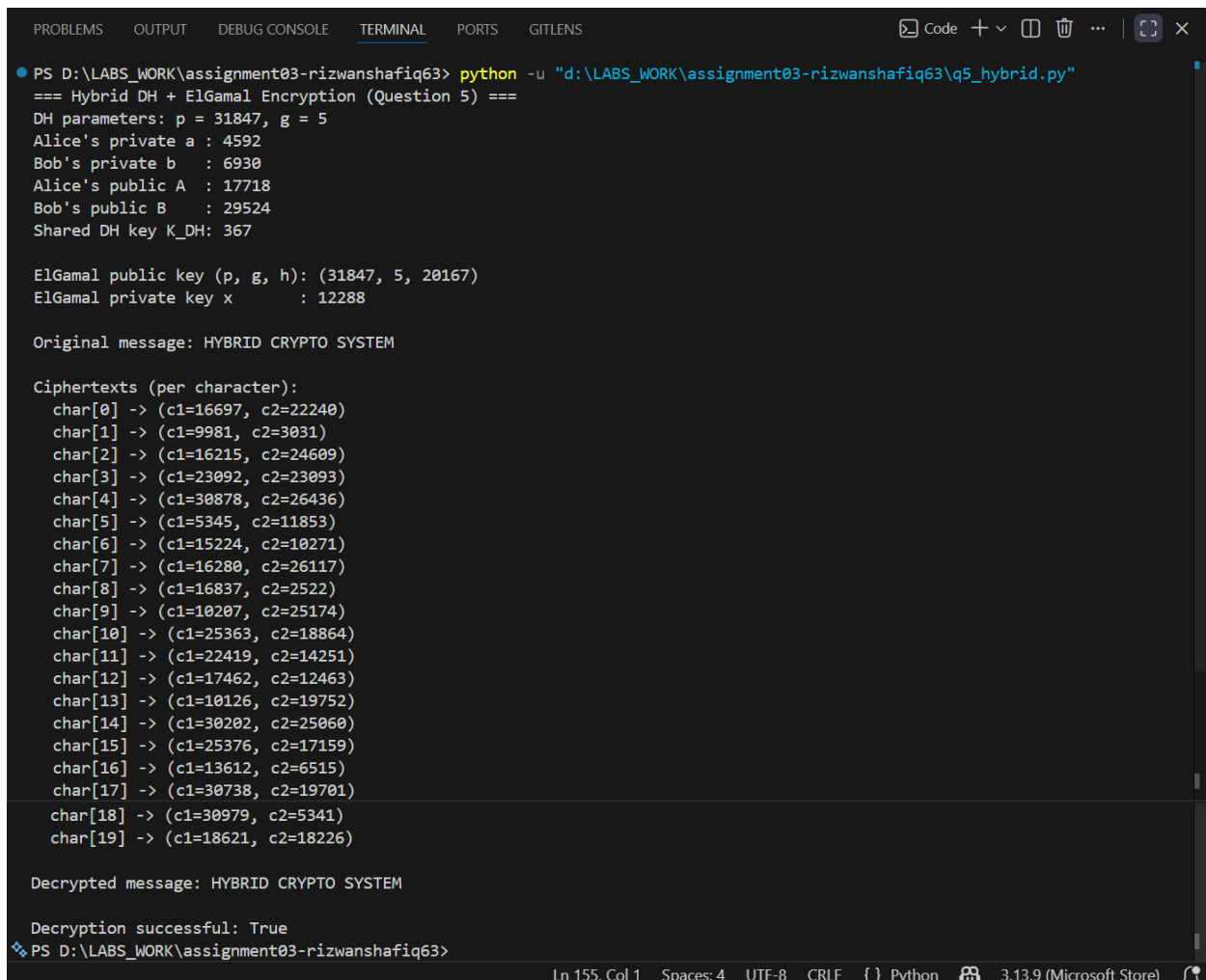
- Used to transmit masked data securely
- Ensures confidentiality even over insecure channels

Outcome

A secure hybrid design that:

- Protects the plaintext with both symmetric and asymmetric layers
- Prevents MITM when DH is authenticated
- Uses randomness + session keys for strong security

Screenshot of Q5 Output:



```
PS D:\LABS_WORK\assignment03-rizwanshafi63> python -u "d:\LABS_WORK\assignment03-rizwanshafi63\q5_hybrid.py"
=== Hybrid DH + ElGamal Encryption (Question 5) ===
DH parameters: p = 31847, g = 5
Alice's private a : 4592
Bob's private b : 6930
Alice's public A : 17718
Bob's public B : 29524
Shared DH key K_DH: 367

ElGamal public key (p, g, h): (31847, 5, 20167)
ElGamal private key x : 12288

Original message: HYBRID CRYPTO SYSTEM

Ciphertexts (per character):
char[0] -> (c1=16697, c2=22240)
char[1] -> (c1=9981, c2=3031)
char[2] -> (c1=16215, c2=24609)
char[3] -> (c1=23092, c2=23093)
char[4] -> (c1=30878, c2=26436)
char[5] -> (c1=5345, c2=11853)
char[6] -> (c1=15224, c2=10271)
char[7] -> (c1=16280, c2=26117)
char[8] -> (c1=16837, c2=2522)
char[9] -> (c1=10207, c2=25174)
char[10] -> (c1=25363, c2=18864)
char[11] -> (c1=22419, c2=14251)
char[12] -> (c1=17462, c2=12463)
char[13] -> (c1=10126, c2=19752)
char[14] -> (c1=30202, c2=25060)
char[15] -> (c1=25376, c2=17159)
char[16] -> (c1=13612, c2=6515)
char[17] -> (c1=30738, c2=19701)
char[18] -> (c1=30979, c2=5341)
char[19] -> (c1=18621, c2=18226)

Decrypted message: HYBRID CRYPTO SYSTEM

Decryption successful: True
PS D:\LABS_WORK\assignment03-rizwanshafi63>
```